1. What is a View in SQL Server?

In **Microsoft SQL Server**, a **View** is a **virtual table** that is created based on the result of a **SELECT query**.

A view **does not store data physically** (except indexed views). Instead, it stores the **SQL query**, and every time the view is used, SQL Server retrieves data from the underlying tables.

Definition

A **View** is a database object that represents the result of a SELECT statement as a virtual table.

Example Tables

Customers

CustomerID	FirstName	City
1	Ramesh	Hyderabad
2	Suresh	Chennai

Accounts

AccountID	CustomerID	Balance
101	1	5000
102	1	10000
103	2	8000

Now suppose we frequently need **customer name and balance** together.

Instead of writing the join every time, we can create a **View**.

---

2. Why Do We Use Views?

Views are used for several practical reasons.

1. Simplify Complex Queries

Complex joins can be saved inside a view.

CREATE VIEW CustomerAccounts

```
AS
SELECT c.FirstName, a.AccountID, a.Balance
FROM Customers c
INNER JOIN Accounts a
ON c.CustomerID = a.CustomerID;
```

Now instead of writing joins repeatedly:

```
SELECT * FROM CustomerAccounts;
```

---

2. Security (Restrict Data Access)

Suppose the **Accounts table contains sensitive data**, but we only want users to see **customer name and balance**.

We create a view that hides other columns.

```
CREATE VIEW CustomerBalance
AS
SELECT c.FirstName, a.Balance
FROM Customers c
JOIN Accounts a
ON c.CustomerID = a.CustomerID;
```

Users access the **view instead of the table**.

---

3. Reusability

Views can be reused in multiple queries.

Example:

```
SELECT * FROM CustomerAccounts
WHERE Balance > 8000;
```

---

4. Data Abstraction

Users **do not need to know table structure**.

Application queries the view without knowing how tables are joined.

---

3. Syntax to Create a View

```
CREATE VIEW ViewName
AS
SELECT column1, column2
FROM TableName
WHERE condition;
```

Example

```
CREATE VIEW HighBalanceAccounts
AS
SELECT AccountID, Balance
FROM Accounts
WHERE Balance > 7000;
```

Querying the view:

```
SELECT * FROM HighBalanceAccounts;
```

Result

AccountID	Balance
102	10000
103	8000

---

4. Types of Views in SQL Server

There are mainly **two types**.

1. Simple View

A view created using **one table**.

Example

```
CREATE VIEW CustomerDetails

AS

SELECT CustomerID, FirstName, City

FROM Customers;
```

Query

```
SELECT * FROM CustomerDetails;
```

---

2. Complex View

A view created using:

- Multiple tables
- Joins
- Aggregate functions
- Group By

Example

```
CREATE VIEW CustomerTotalBalance

AS

SELECT

   c.FirstName,

   SUM(a.Balance) AS TotalBalance

FROM Customers c

JOIN Accounts a

ON c.CustomerID = a.CustomerID

GROUP BY c.FirstName;
```

Query

```
SELECT * FROM CustomerTotalBalance;
```

Result

FirstName	TotalBalance
Ramesh	15000
Suresh	8000

---

5. How to View Existing Views

To list views in a database:

```
SELECT *
FROM sys.views;
```

---

6. Updating a View

To modify an existing view.

```
ALTER VIEW CustomerAccounts
AS
SELECT c.FirstName, c.City, a.Balance
FROM Customers c
JOIN Accounts a
ON c.CustomerID = a.CustomerID;
```

---

7. Dropping a View

To delete a view.

```
DROP VIEW CustomerAccounts;
```

---

8. Using Views for Insert, Update, Delete

Some views allow **DML operations**.

Insert

```
INSERT INTO CustomerDetails(CustomerID, FirstName, City)

VALUES (3, 'Mahesh', 'Mumbai');
```

Update

```
UPDATE CustomerDetails

SET City = 'Bangalore'

WHERE CustomerID = 1;
```

Delete

```
DELETE FROM CustomerDetails

WHERE CustomerID = 2;
```

⚠️ These work only if the view follows rules:

- Based on **single table**
- No **GROUP BY**
- No **aggregate functions**
- No **DISTINCT**

---

9. WITH CHECK OPTION

This ensures **inserted or updated rows satisfy the view condition**.

Example:

```
CREATE VIEW HighBalance

AS

SELECT *

FROM Accounts

WHERE Balance > 7000
```

WITH CHECK OPTION;

Now this will fail:

INSERT INTO HighBalance VALUES (104,1,5000);

Because **5000 does not satisfy the view condition**.

---

11. Advantages of Views

1. Simplifies complex queries
2. Improves security
3. Provides logical data abstraction
4. Reusable query logic
5. Can improve performance (Indexed Views)

---

12. Disadvantages of Views

1. Performance overhead for complex views
2. Cannot always perform INSERT/UPDATE/DELETE
3. Nested views may reduce readability

---

13. Real-World Example (Banking Scenario)

Suppose a bank wants to show **customer balances** to employees.

But hide:

- AccountID
- CustomerID

Create a view:

CREATE VIEW CustomerBalanceReport

AS

```
SELECT
    c.FirstName,
    SUM(a.Balance) AS TotalBalance
FROM Customers c
JOIN Accounts a
ON c.CustomerID = a.CustomerID
GROUP BY c.FirstName;
```

Employees query:

```
SELECT * FROM CustomerBalanceReport;
```

They **cannot see underlying tables**.